IAM-10: Templated Policy-Group Assignments

> **Series:** IAM | **Notebook:** 10 of 10 | **Created:** February 2026 | **Last Updated:** 03/05/2026

Overview

Parameterized policies eliminate the need to duplicate policies for every team, bucket, or security context. Instead of creating 20 nearly-identical policies for 20 teams, you create **one policy template** with `${bindParam:...}` placeholders and supply different values when binding the policy to each group.

This notebook walks you through the complete lifecycle: template syntax, UI creation, IAM API binding, Monaco automation, common patterns, DQL audit queries, and troubleshooting.

Table of Contents

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Gen3 IAM enabled
Permissions	`account-iam-admin` to create/modify policies and bindings
API Access	OAuth client or API token with IAM management scopes
Prior Knowledge	**IAM-04: Policy Authoring and Management** (policy syntax, conditions)
Optional	Monaco CLI installed (for policy-as-code section)

1. Why Policy Templates?

The Problem: Policy Sprawl

Without templates, every team that needs scoped data access requires its own dedicated policy. For an organization with 15 teams, that means:

- **15 nearly-identical policies** to create
- **15 policies to maintain** when permissions change
- **15 policies to audit** during compliance reviews

The Solution: Parameterized Policies

With templates, you create **one policy** with `${bindParam:...}` placeholders and create **15 bindings** with different parameter values:

Approach	Policies	Bindings	Maintenance Points	Audit Scope
Without Templates	15	15	15 policies to update	15 policies to review
With Templates	1	15	1 policy to update	1 template + bindings

When to Use Templates vs Static Policies

Scenario	Recommendation
Multiple teams need identical permissions with different scopes	**Use template**
Single team with unique requirements	Static policy
Environment-based access (prod/staging/dev)	**Use template**
One-off admin or auditor policy	Static policy
Large-scale MZ migration (many MZs)	**Use template** (see **MZ2POL-08**)

2. Template Syntax Deep Dive

The `${bindParam:...}` Placeholder

Bind parameters are placeholders in the **WHERE condition** of a policy statement. They are replaced with actual values when the policy is bound to a group.

Syntax: `${bindParam:parameter-name}`

Rules:
- Parameters appear **only** in WHERE condition values
- Parameter names are **case-sensitive**
- Names should use alphanumeric characters and hyphens

Single Parameter Example

```
ALLOW storage:logs:read WHERE storage:dt.security_context = "${bindParam:team}";
```

When bound with `{"parameters": {"team": "checkout"}}`, this resolves to:

```
ALLOW storage:logs:read WHERE storage:dt.security_context = "checkout";
```

Multiple Parameters Example

```
ALLOW storage:logs:read
  WHERE storage:dt.security_context = "${bindParam:team}"
  AND storage:bucket.name = "${bindParam:bucket}";
```

Binding: `{"parameters": {"team": "checkout", "bucket": "checkout_logs"}}`

Multi-Statement Template

A single template can grant access across multiple data types using the same parameter:

```
ALLOW storage:logs:read WHERE storage:dt.security_context = "${bindParam:team}";
ALLOW storage:spans:read WHERE storage:dt.security_context = "${bindParam:team}";
ALLOW storage:metrics:read WHERE storage:dt.security_context = "${bindParam:team}";
ALLOW settings:objects:read WHERE settings:dt.security_context = "${bindParam:team}";
```

All four statements resolve using the **same** `team` parameter value from the binding.

Parameter Scope Reference

Field	Supports bindParam	Domain
`storage:dt.security_context`	Yes	Storage (logs, spans, metrics)
`storage:bucket.name`	Yes	Storage (bucket-level)
`settings:dt.security_context`	Yes	Settings objects
`settings:schemaId`	Yes	Settings schema
`environment:management-zone`	Yes	Entity access

3. Creating a Parameterized Policy via UI

Step-by-Step

1. Navigate to **Account Management** → **Identity & Access Management** → **Policies**
2. Click **Create policy**
3. Enter the policy name using the naming convention: `tpl-<scope>-<permission>`
4. Enter a description that mentions it is parameterized (e.g., "Parameterized: read access scoped to team security context")
5. Paste the parameterized policy statement with `${bindParam:...}` placeholders
6. Click **Save**

Dynatrace automatically recognizes the bind parameters and will prompt for their values when the policy is bound to a group.

Naming Convention for Template Policies

Convention	Example	Description
`tpl-team-data-reader`	Team-scoped read access	Read logs, spans, metrics by security context
`tpl-team-data-editor`	Team-scoped read+write	Full data access by security context
`tpl-env-data-reader`	Environment-scoped read	Read access by environment
`tpl-bucket-reader`	Bucket-scoped read	Read access by bucket name
`tpl-full-three-domain`	Full MZ replacement	All three domains scoped by parameter

Verification

After saving, the policy should appear in the policy list. When you bind it to a group, the UI will display input fields for each `${bindParam:...}` parameter.

4. Binding with Parameters via IAM API

The UI supports binding, but the **IAM API** enables automation at scale — essential when binding the same template to dozens of groups.

API Endpoints

Operation	Method	Endpoint
List policies	GET	`/iam/v1/repo/{levelType}/{levelId}/policies`
List bindings	GET	`/iam/v1/repo/{levelType}/{levelId}/bindings/{policyUuid}`
Create binding	POST	`/iam/v1/repo/{levelType}/{levelId}/bindings/{policyUuid}/{groupUuid}`
Delete binding	DELETE	`/iam/v1/repo/{levelType}/{levelId}/bindings/{policyUuid}/{groupUuid}`

Authentication: OAuth2 bearer token or API token with IAM management scopes.

Step 1: Find Policy UUID

```
curl -X GET \
  'https://api.dynatrace.com/iam/v1/repo/account/&lt;ACCOUNT_UUID&gt;/policies' \
  -H 'Authorization: Bearer &lt;TOKEN&gt;'
```

Look for your template policy name (e.g., `tpl-team-data-reader`) in the response and note its `uuid`.

Step 2: Find Group UUID

```
curl -X GET \
  'https://api.dynatrace.com/iam/v1/accounts/&lt;ACCOUNT_UUID&gt;/groups' \
  -H 'Authorization: Bearer &lt;TOKEN&gt;'
```

Step 3: Create Binding with Parameters

```
curl -X POST \

'https://api.dynatrace.com/iam/v1/repo/account/&lt;ACCOUNT_UUID&gt;/bindings/&lt;POLICY_UU
ID&gt;/&lt;GROUP_UUID&gt;' \
  -H 'Authorization: Bearer &lt;TOKEN&gt;' \
  -H 'Content-Type: application/json' \
  -d '{"parameters": {"team": "checkout"}}'
```

Step 4: Verify Binding

```
curl -X GET \

'https://api.dynatrace.com/iam/v1/repo/account/&lt;ACCOUNT_UUID&gt;/bindings/&lt;POLICY_UU
ID&gt;' \
  -H 'Authorization: Bearer &lt;TOKEN&gt;'
```

Bulk Binding: Multiple Teams

Bind the same template to multiple groups with different parameter values:

```
#!/bin/bash
ACCOUNT_UUID="&lt;your-account-uuid&gt;"
POLICY_UUID="&lt;team-reader-template-uuid&gt;"
TOKEN="&lt;bearer-token&gt;"
API_BASE="https://api.dynatrace.com/iam/v1/repo/account/${ACCOUNT_UUID}"

declare -A TEAM_GROUPS=(
  ["checkout"]="&lt;group-uuid-checkout&gt;"
  ["payments"]="&lt;group-uuid-payments&gt;"
  ["catalog"]="&lt;group-uuid-catalog&gt;"
  ["search"]="&lt;group-uuid-search&gt;"
  ["frontend"]="&lt;group-uuid-frontend&gt;"
)

for team in "${!TEAM_GROUPS[@]}"; do
  GROUP_UUID="${TEAM_GROUPS[$team]}"
  echo "Binding template to ${team} (${GROUP_UUID})..."
  curl -s -X POST \
    "${API_BASE}/bindings/${POLICY_UUID}/${GROUP_UUID}" \
    -H "Authorization: Bearer ${TOKEN}" \
    -H 'Content-Type: application/json' \
    -d "{\"parameters\": {\"team\": \"${team}\"}}" \
    -o /dev/null -w "  HTTP %{http_code}\n"
done
```

> **Tip:** A 201 response means the binding was created. A 409 means it already exists.
>
> **Full working example:** For a complete script that creates groups, policies across all three domains (UI + Data + Config), and binds everything together for a four-persona model, see the **End-to-End Provisioning Script** in **IAM-11: Policy Persona Workshop**.

5. Monaco YAML for Policy-as-Code

Version-controlled, peer-reviewed, CI/CD-deployed IAM configuration.

Project Structure

```
iam-config/
├── manifest.yaml
├── policies/
│   ├── tpl-team-data-reader.yaml
│   └── tpl-team-data-editor.yaml
├── groups/
│   └── team-groups.yaml
└── bindings/
    └── team-bindings.yaml
```

Policy Template YAML

```
# policies/tpl-team-data-reader.yaml
configs:
  - id: tpl-team-data-reader
    type:
      settings:
        schema: builtin:iam.policy
    config:
      name: "tpl-team-data-reader"
      description: "Parameterized: read access scoped to ${bindParam:team}"
      statements:
        - &gt;-
          ALLOW storage:logs:read
          WHERE storage:dt.security_context = "${bindParam:team}";
        - &gt;-
          ALLOW storage:spans:read
          WHERE storage:dt.security_context = "${bindParam:team}";
        - &gt;-
          ALLOW storage:metrics:read
          WHERE storage:dt.security_context = "${bindParam:team}";
```

Binding YAML with Parameters

```
# bindings/team-bindings.yaml
configs:
  - id: checkout-team-binding
    type:
      settings:
        schema: builtin:iam.binding
    config:
      policyId: "{{ .tpl-team-data-reader }}"
      groupId: "{{ .dt-checkout-editors }}"
      parameters:
        team: "checkout"

  - id: payments-team-binding
    type:
      settings:
        schema: builtin:iam.binding
    config:
      policyId: "{{ .tpl-team-data-reader }}"
      groupId: "{{ .dt-payments-editors }}"
      parameters:
        team: "payments"
```

GitOps Workflow

1. **Branch** — Create feature branch for IAM change
2. **Edit** — Modify policy or binding YAML

3. **Review** — Submit PR, get peer review from IAM admin
4. **Test** — Deploy to test environment with `monaco deploy`
5. **Approve** — Merge after validation
6. **Deploy** — CI/CD applies to production

> **Note:** IAM binding support in Monaco may vary by version. Always verify against the Monaco documentation. See also **AUTOM-03: Monaco** for full automation workflow.

6. Common Template Patterns

Pattern 1: Team-Scoped Data Access

The most common pattern — scope all data access by team security context.

```
ALLOW storage:logs:read WHERE storage:dt.security_context = "${bindParam:team}";
ALLOW storage:spans:read WHERE storage:dt.security_context = "${bindParam:team}";
ALLOW storage:metrics:read WHERE storage:dt.security_context = "${bindParam:team}";
```

Binding: `{"parameters": {"team": "checkout"}}` or `{"parameters": {"team": "payments"}}`

Pattern 2: Environment-Scoped Access

Scope access by environment (prod, staging, dev).

```
ALLOW storage:*:read WHERE storage:dt.security_context = "${bindParam:environment}";
ALLOW settings:objects:read WHERE settings:dt.security_context =
"${bindParam:environment}";
```

Binding: `{"parameters": {"environment": "prod"}}` or `{"parameters": {"environment": "staging"}}`

Pattern 3: Bucket-Scoped Team Isolation

Hard data isolation using dedicated buckets per team.

```
ALLOW storage:logs:read WHERE storage:bucket.name = "${bindParam:log-bucket}";
ALLOW storage:spans:read WHERE storage:bucket.name = "${bindParam:span-bucket}";
ALLOW storage:metrics:read WHERE storage:bucket.name = "${bindParam:metric-bucket}";
```

Binding: `{"parameters": {"log-bucket": "checkout_logs", "span-bucket": "checkout_spans", "metric-bucket": "checkout_metrics"}}`

Pattern 4: Combined Team + Environment

Defense-in-depth: require both team and bucket match.

```
ALLOW storage:*:read
  WHERE storage:dt.security_context = "${bindParam:team}"
  AND storage:bucket.name = "${bindParam:bucket}";
```

Binding: `{"parameters": {"team": "checkout", "bucket": "prod_checkout_logs"}}`

Pattern 5: Settings Schema Scoped

Grant access to specific settings categories.

```
ALLOW settings:objects:* WHERE settings:schemaId startsWith "${bindParam:schema-prefix}";
```

Binding: `{"parameters": {"schema-prefix": "builtin:alerting"}}` for the alerting team

Pattern 6: Full Three-Domain (MZ Replacement)

Complete replacement of a Management Zone — covers entity, data, and settings access.

```
ALLOW storage:*:read WHERE storage:dt.security_context = "${bindParam:scope}";
ALLOW storage:*:write WHERE storage:dt.security_context = "${bindParam:scope}";
ALLOW settings:objects:* WHERE settings:dt.security_context = "${bindParam:scope}";
```

Combined with a **boundary** using the same security context for entity-level restriction.

Binding: `{"parameters": {"scope": "LOB5"}}` or `{"parameters": {"scope": "team-frontend"}}`

> **For MZ migration at scale**, see **MZ2POL-08: Templated Policies for MZ Migration** which covers converting MZ patterns to these templates.

Pattern Summary

Pattern	Parameters	Use Case
Team-Scoped	`team`	Team data isolation via security context
Environment-Scoped	`environment`	Environment-based access control
Bucket-Scoped	`log-bucket`, `span-bucket`, `metric-bucket`	Hard data isolation via buckets
Combined	`team`, `bucket`	Defense-in-depth (team + bucket)
Settings Schema	`schema-prefix`	Capability-based settings access
Three-Domain	`scope`	Full MZ replacement

7. DQL Audit Queries

Use these queries to verify that templated policy bindings are working correctly and to audit policy-related changes.

```
// Track recent policy binding changes in audit log
fetch logs, from: now() - 30d
| filter matchesPhrase(log.source, "audit")
| filter matchesPhrase(content, "binding") or matchesPhrase(content, "policy")
| filter matchesPhrase(content, "created") or matchesPhrase(content, "modified")
| fields timestamp, content
| sort timestamp desc
| limit 50
```

```
// Find access denied events that may indicate parameter misconfiguration
fetch logs, from: now() - 7d
| filter matchesPhrase(log.source, "audit")
| filter matchesPhrase(content, "denied") or matchesPhrase(content, "forbidden")
| fields timestamp, content
| sort timestamp desc
| limit 100
```

```
// Verify security context coverage — entities need context for template binding to work
fetch dt.entity.service
| summarize
    total = count(),
    withContext = countIf(isNotNull(dt.security_context)),
    missingContext = countIf(isNull(dt.security_context))
| fields total, withContext, missingContext,
         coveragePercent = round(100.0 * withContext / total, decimals: 2)
```

```
// Verify security context values match expected team parameters
fetch dt.entity.service
| filter isNotNull(dt.security_context)
| summarize count = count(), by:{dt.security_context}
| sort count desc
```

8. Troubleshooting Templated Policies

Common Issues

Issue	Symptom	Cause	Resolution
Parameter not resolved	Access denied despite binding	Parameter name mismatch (case-sensitive)	Verify parameter name matches exactly between policy and binding
Binding not applied	No access at all	Missing binding POST call	Verify binding exists via GET bindings endpoint
Wrong scope	User sees too much or too little	Parameter value incorrect	Check binding parameter values via API
Template change not reflected	Old behavior persists	Caching / eventual consistency	Wait 5 minutes, then re-verify
Multiple parameters misaligned	Partial access	One parameter correct, another wrong	Check all parameter values in binding response

Debugging Checklist

1. **Verify policy exists** and contains `${bindParam:...}` syntax
2. **Verify group UUID** is correct (list groups via API)
3. **Verify binding exists** (GET bindings endpoint — check response includes your group)
4. **Verify parameter values** in the binding response match expected values
5. **Verify security context** on target entities matches the parameter value
6. **Test with admin account** to confirm the data exists
7. **Check audit logs** for denied events (use DQL queries above)

> **Tip:** Use the GET bindings endpoint to dump all bindings for a policy and programmatically compare parameter values against expected values. This catches typos and mismatches quickly.

For comprehensive IAM troubleshooting, see **IAM-09: Troubleshooting Access Issues**.

Summary

In this notebook, you learned:

- Why policy templates reduce policy sprawl and simplify management
- The `${bindParam:parameter-name}` syntax and where it can be used
- How to create parameterized policies via the Dynatrace UI
- How to bind policies with parameters using the IAM API (including bulk operations)
- Monaco YAML patterns for policy-as-code with templates
- Six common template patterns for team, environment, and bucket scoping
- DQL audit queries to verify bindings are working
- Troubleshooting methodology for parameterized policies

Next Steps

- **IAM-05: Boundary Design Patterns** — Combine templated policies with boundaries for defense in depth
- **IAM-07: Audit Logging and Compliance** — Set up ongoing audit monitoring for policy bindings
- **AUTOM-03: Monaco** — Full Monaco configuration automation workflow
- **MZ2POL-08: Templated Policies for MZ Migration** — Apply templates to MZ migration at scale
- **IAM-11: Policy Persona Workshop** — Map organizational personas to policy templates using a structured design methodology. Design your persona model, then use **IAM-12** for the provisioning scripts that creates groups, policies (UI + Data + Config), and templated bindings for a four-persona model via the IAM API.

References

- Policy Templating
- Policy Management API
- Working with Policies
- IAM Policy Statement Syntax